ERASMUS INTERNSHIP TECHNICAL REPORT

Ansible-Based Linux/Ubuntu Lab Management Toolkit

Project Focus

Centralized, safe and professor-friendly management of Ubuntu student laboratory computers using Ansible over SSH.

Prepared by: Farid Nowrouzi
Erasmus+ Traineeship / Department Placement
Department of Physics, Faculty of Natural Sciences, University of Tirana
Prepared for: Prof. As. Dr. Klaudio Peqini
Date: July 2026

Repository: ansible-ubuntu-lab-management

Report Information

Field	Value
Project title	Ansible-Based Linux/Ubuntu Lab Management Toolkit
Report type	Erasmus Internship Final Technical Report
Author	Farid Nowrouzi
Host context	University computer laboratory management
Primary technology	Ansible over SSH on Ubuntu/Linux systems
Main deliverable	A documented toolkit for managing student lab PCs from a teacher/control computer
Repository	Farid-Nowrouzi/ansible-ubuntu-lab-management
License	MIT License
Validation status	Implementation complete; real lab validation recorded separately in FINAL_TEST_REPORT.md

Important Note on Validation

This report documents the implemented toolkit and the validation methodology. Final real-machine results should be recorded in FINAL_TEST_REPORT.md after testing on the Ubuntu control node and student PCs.

Table of Contents

1. Executive Summary

This report presents an Ansible-based Linux/Ubuntu Lab Management Toolkit developed as part of an Erasmus internship. The project addresses a practical laboratory administration problem: managing several Ubuntu student computers from one teacher/control computer without repeating manual work on every machine.

The toolkit provides a structured repository with Ansible playbooks, an interactive professor-friendly launcher, centralized configuration, private inventory management, controlled file distribution, logging, testing documentation and handover guides. Its design emphasizes safety, clarity, reproducibility and long-term maintainability.

The central idea is simple: the professor runs ./labmanage, selects an action, chooses the target PCs, and the toolkit runs the appropriate Ansible playbook while saving logs into the reports/ directory. Routine changes are made through inventory.ini, config/lab_settings.yml and shared_materials/, while the playbooks remain the automation layer.

Main Outcome	Description
Centralized management	Student PCs can be managed from one teacher/control computer over SSH.
Safe workflow	Preflight checks, confirmations and one-PC-first testing reduce operational risk.
Professor usability	The ./labmanage launcher provides a simple menu instead of requiring long Ansible commands.
Maintainability	Configuration is separated from logic through config/lab_settings.yml.
Documentation	Guides are provided for setup, customization, troubleshooting, handover and final testing.

Final Project Statement

The toolkit is designed as a realistic, practical and documented Erasmus internship deliverable rather than a complex enterprise management platform.

2. Internship Context and Problem Statement

The internship involved practical technical work connected to computer laboratory administration. The lab environment consists of a teacher/control computer and multiple Linux/Ubuntu student computers. Without centralized management, a professor or administrator must perform repetitive tasks manually on each machine.

Typical repetitive tasks include updating operating systems, installing teaching software, copying exercises or datasets, checking whether machines are reachable, collecting basic status information, cleaning unnecessary package files and rebooting machines when updates require it.

Manual administration is time-consuming and can create inconsistent machine states. Some computers may be updated, while others remain outdated. Some may receive new teaching material, while others may not. This creates operational risk before lessons and laboratory sessions.

Problem Statement

Problem to Solve

How can a teacher/control computer centrally manage Ubuntu student computers in a safe, repeatable and documented way without manually repeating the same work on every machine?

- The solution should be understandable for future maintainers.
- The solution should avoid unnecessary enterprise complexity.
- The solution should keep private lab information out of the public repository.
- The solution should include documentation and a clear testing workflow.

3. Objectives and Scope

The project was intentionally scoped as a practical toolkit rather than a full enterprise platform. The first working version focuses on core laboratory management tasks that are useful immediately in a small Ubuntu lab environment.

Objective	Implementation
Check connectivity	01_check_connection.yml verifies that managed PCs are reachable.
Validate readiness	00_preflight_check.yml checks inventory, SSH, Python, sudo, OS, disk and memory.
Collect status	02_collect_lab_status.yml gathers useful system information.
Update systems	03_update_system.yml manages Ubuntu package updates.
Install software	04_install_required_software.yml installs packages from config/lab_settings.yml.
Copy teaching files	05_copy_shared_materials.yml distributes approved files from shared_materials/.
Clean lab computers	06_clean_lab_computers.yml performs conservative cleanup actions.
Reboot if required	07_reboot_if_required.yml reboots only when Ubuntu marks reboot as required.
Simplify usage	./labmanage launches an interactive menu for professor-friendly operation.
Save evidence	run_with_logging.sh writes terminal output into reports/.

Out of Scope

- Docker-based teaching environments
- Prometheus/Grafana monitoring dashboards
- LDAP/Active Directory integration
- Full imaging/cloning systems such as FOG or Clonezilla
- Enterprise compliance automation
- CI/CD pipelines and Molecule tests

Scope Decision

The project remains intentionally lightweight because the purpose is to deliver a practical internship result that can be understood, tested and maintained in the department.

4. Requirements Analysis

The requirements were derived from the practical needs of a university Ubuntu laboratory. The system should be safe to run, easy to understand, documented and adaptable to changing teaching requirements.

Requirement	Reason	Project Response
Central control	Avoid repeated manual work on each computer.	Ansible runs from the teacher/control computer.
Inventory-based targeting	Manage specific PCs or the whole lab.	inventory.ini defines the students group and host entries.
Safe preflight checks	Prevent risky operations when machines are not ready.	00_preflight_check.yml runs before changing actions.
Easy interface	Professor should not remember long commands.	./labmanage opens a guided menu.
Configurable packages/settings	Routine changes should not require editing playbooks.	config/lab_settings.yml holds common settings.
Controlled file sharing	Avoid exposing private professor files.	Only shared_materials/ is copied.
Testing evidence	Demonstrate what was run and what happened.	reports/ stores logs and FINAL_TEST_REPORT.md records validation.

5. System Architecture

The architecture separates user interaction, configuration, inventory, automation and reporting. This separation makes the project easier to explain and safer to maintain.

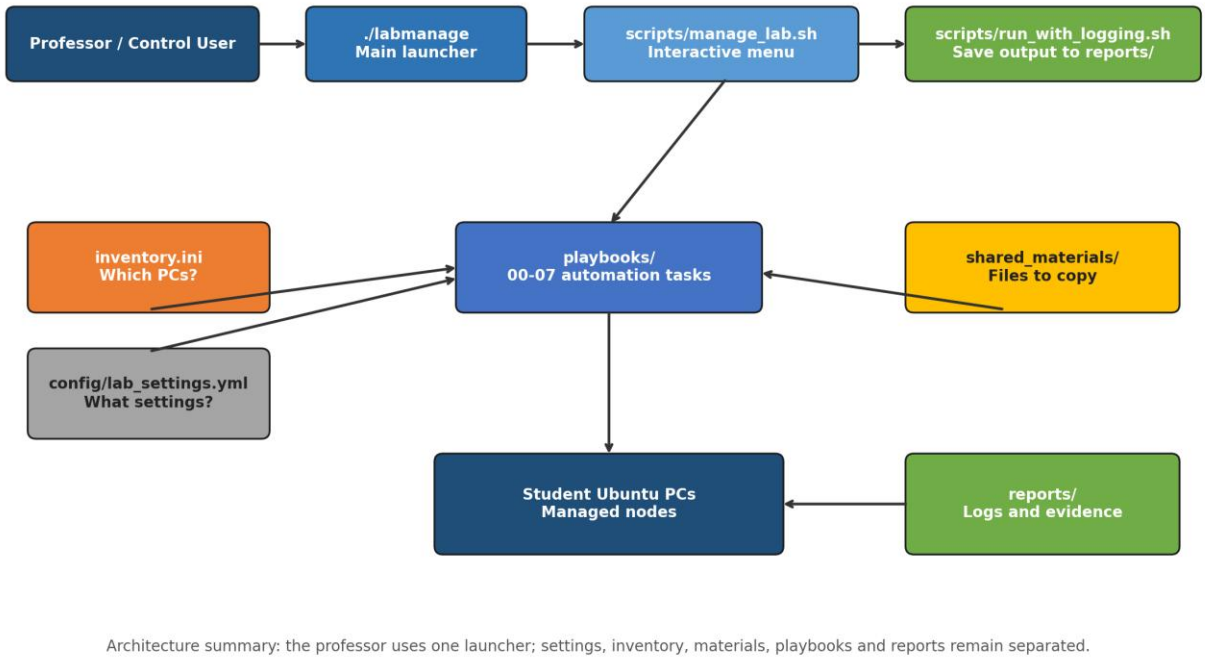

Figure 1. High-level architecture of the Linux/Ubuntu Lab Management Toolkit.

Layer	Purpose
Professor interface	./labmanage and scripts/manage_lab.sh provide the guided user interface.
Logging layer	scripts/run_with_logging.sh runs playbooks and saves output to reports/.
Configuration layer	config/lab_settings.yml controls packages, thresholds and behavior.
Inventory layer	inventory.ini defines which student PCs are managed.
Automation layer	playbooks/ contains Ansible tasks for each operation.
Managed nodes	Student Ubuntu PCs receive commands through SSH.

6. Repository Structure

The repository is organized to separate code, documentation, configuration, controlled teaching materials and generated reports.

```
ansible-ubuntu-lab-management/
├── labmanage
├── ansible.cfg
├── inventory.example.ini
├── inventory.ini                 # private local file, not committed
├── config/
│   └── lab_settings.yml
├── playbooks/
│   ├── 00_preflight_check.yml
│   ├── 01_check_connection.yml
│   ├── 02_collect_lab_status.yml
│   ├── 03_update_system.yml
│   ├── 04_install_required_software.yml
│   ├── 05_copy_shared_materials.yml
│   ├── 06_clean_lab_computers.yml
│   └── 07_reboot_if_required.yml
├── scripts/
│   ├── manage_lab.sh
│   └── run_with_logging.sh
├── shared_materials/
├── reports/
├── docs/
├── README.md
├── LAB_TEST_PLAN.md
├── FINAL_TEST_REPORT.md
├── PROJECT_STATUS.md
└── CHANGELOG.md
```

Path	Description
labmanage	Root-level launcher used by the professor.
config/lab_settings.yml	Central settings file for normal lab behavior.
inventory.ini	Private list of real lab PCs and SSH users.
inventory.example.ini	Public safe template for inventory structure.
playbooks/	Ansible automation files.
scripts/	Menu and logging helper scripts.
shared_materials/	Controlled folder for teaching files to copy.
reports/	Generated logs and validation evidence.
docs/	Setup, handover, customization and troubleshooting guides.

7. Core Components

7.1 Professor Launcher: labmanage

The root-level labmanage script is the main entry point for the professor. It detects the project structure and starts the interactive menu located in scripts/manage_lab.sh. It allows the professor to start the toolkit with one simple command.

```
./labmanage
```

7.2 Interactive Menu: scripts/manage_lab.sh

The menu displays the main available operations, asks for target PCs, handles authentication prompts, confirms potentially disruptive actions and returns to the menu after each run. It also supports friendly input such as q, quit or exit.

7.3 Logging Wrapper: scripts/run_with_logging.sh

The logging wrapper runs Ansible playbooks while saving terminal output into timestamped files under reports/. This provides evidence of what was executed during testing and maintenance.

7.4 Central Configuration: config/lab_settings.yml

The configuration file contains normal lab settings such as packages to install, shared-materials destination, disk thresholds, cleanup behavior and reboot timeout. This allows routine customization without editing Ansible logic.

7.5 Private Inventory: inventory.ini

The private inventory lists the real student PCs, IP addresses or hostnames, SSH usernames and connection details. It is intentionally ignored by Git because it may contain private lab information.

8. Playbooks Implemented

Playbook	Purpose	Safety Level
00_preflight_check.yml	Read-only readiness check before maintenance actions.	Safe / read-only
01_check_connection.yml	Checks whether Ansible can reach managed nodes.	Safe / read-only
02_collect_lab_status.yml	Collects hostname, OS, disk, memory and uptime information.	Safe / read-only
03_update_system.yml	Runs configured Ubuntu package update behavior.	Changing
04_install_required_software.yml	Installs packages listed in config/lab_settings.yml.	Changing
05_copy_shared_materials.yml	Copies approved files from shared_materials/ to student PCs.	Changing
06_clean_lab_computers.yml	Performs conservative cleanup operations.	Changing

07_reboot_if_required.yml	Reboots only if Ubuntu indicates reboot is required.	Potentially disruptive

Operational Rule

Changing playbooks should be tested on one PC first, then a small group, and only then the full lab.

9. Professor Workflow and Usability

The final workflow was designed around a simple question: what should the professor actually do when using the project? The answer is intentionally short: run ./labmanage and follow the menu.

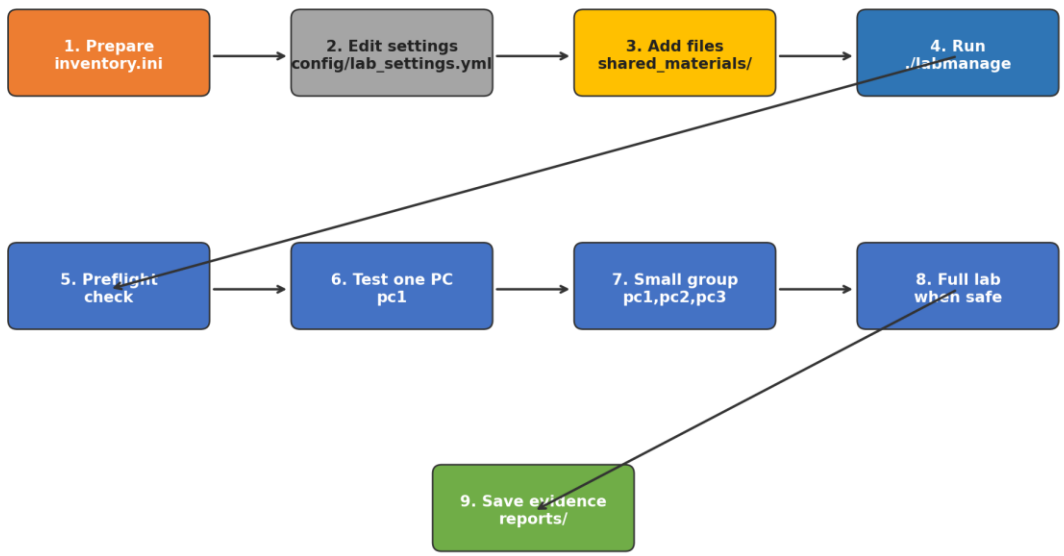

Figure 2. Recommended professor workflow and validation progression.

User Need	What to Use
Run the system	./labmanage
Add or remove PCs	inventory.ini
Change packages/settings	config/lab_settings.yml
Add teaching files	shared_materials/
Review results	reports/
Change automation logic	playbooks/ - technical maintainers only

10. Configuration and Customization

Configuration was separated from automation logic to make the toolkit easier to maintain. Instead of editing playbooks directly, normal changes are performed in config/lab_settings.yml.

```
# Example: packages installed by 04_install_required_software.yml
required_packages:
  - vim
  - git
  - curl
  - wget
  - python3
  - python3-pip
  - htop
  - tree

# Example: shared teaching materials destination
shared_materials_destination: "/home/student/Lab_Materials"

# Example: preflight thresholds
minimum_free_disk_gb: 2
minimum_memory_mb_warning: 2048
```

This design provides a clean mental model: inventory.ini controls which machines are managed, config/lab_settings.yml controls what behavior is used, shared_materials/ contains files to distribute, and playbooks/ implement the automation.

11. Safety, Security and Privacy

The toolkit includes several safeguards to reduce risk during laboratory operation. Safety is especially important because update, cleanup and reboot actions can affect active users.

Safety Feature	Explanation
Preflight check	Verifies connectivity, OS, Python, sudo, disk and memory before serious actions.
Menu confirmations	Changing actions require confirmation; reboot requires stronger confirmation.
One-PC-first testing	Documentation and menu guidance recommend testing pc1 before wider deployment.
Private inventory	inventory.ini is ignored by Git to protect real IPs and usernames.
Controlled materials folder	Only shared_materials/ is copied; private professor files are not shared.
Generated reports ignored	Logs may contain private hostnames or IP addresses and are kept local.

MIT License	The repository has clear reuse terms for future students and maintainers.

Do Not Run During Class

Update, install, cleanup and reboot playbooks should not be run during active teaching sessions unless the professor explicitly accepts the interruption risk.

12. Testing and Validation Methodology

Testing is a central part of this project. The correct validation sequence moves from syntax checks to one-machine testing, then to a small group, and finally to all reachable machines.

Phase	Validation Step	Expected Result
1	Check Bash script syntax	No shell syntax errors.
2	Check Ansible playbook syntax	All playbooks parse successfully.
3	Run preflight on pc1	pc1 passes readiness checks.
4	Run connection check on pc1	Ansible reaches pc1 successfully.
5	Collect status from pc1	Hostname, OS, disk, memory and uptime appear.
6	Copy test material to pc1	File exists on remote PC and content matches.
7	Install package on pc1	Selected package is installed and verified.
8	Run logging wrapper	A timestamped report file appears in reports/.
9	Repeat safe checks on small group	Reachable PCs pass; failures are clear.
10	Run safe checks on full lab	Lab-wide readiness and connection state is known.

13. Expected Validation Evidence

Validation should produce concrete evidence. The project includes FINAL_TEST_REPORT.md for recording results, and run_with_logging.sh for saving terminal output into reports/.

Evidence Type	Where It Is Stored
Command logs	reports/*.log
Final validation summary	FINAL_TEST_REPORT.md
Screenshots	Optional, stored outside Git if they contain private information.
GitHub repository	Public source and documentation excluding private inventory.
Test file verification	Ansible stat/cat command output or physical PC verification.

```
# Example verification for copied teaching material
ansible pc1 -i inventory.ini -m stat -a
"path=/home/student/Lab_Materials/test_ansible_material.txt" --ask-pass

# Example verification of file content
ansible pc1 -i inventory.ini -m command -a "cat
/home/student/Lab_Materials/test_ansible_material.txt" --ask-pass
```

14. Handover and Maintenance

A successful handover requires that the professor or a future student can understand the project quickly and perform routine operations safely. The documentation folder provides guides for setup, customization, adding new PCs, project overview, troubleshooting and professor handover.

Document	Purpose
README.md	Main repository introduction and quick overview.
docs/quick_start.md	Fast start commands for initial usage.
docs/professor_handover.md	Main professor-facing handover guide.
docs/customization_guide.md	Explains config/lab_settings.yml and routine settings.
docs/adding_new_pc.md	Explains how to add a new student computer.

docs/project_overview.md	High-level architecture and workflow overview.
docs/troubleshooting.md	Common issues and fixes.
FINAL_TEST_REPORT.md	Template for real lab validation results.

15. Scope Boundaries and Design Decisions

The current toolkit was intentionally designed as a practical first-version system for managing Ubuntu laboratory computers from a central control machine. The focus was not to build a complex enterprise platform, but to deliver a safe, understandable, documented, and immediately usable solution for the laboratory environment.

Several design decisions were made deliberately. The inventory is manually maintained to keep the system transparent and controlled. The playbooks are kept separate instead of being converted into advanced Ansible roles so that the structure remains easier to understand, maintain, and teach. The menu-based launcher was added to make the system usable by a professor or maintainer without requiring deep Ansible knowledge.

More advanced infrastructure features, such as monitoring dashboards, automatic machine discovery, scheduling, or imaging systems, were left outside the scope of this deliverable. This boundary keeps the delivered project focused on the requested laboratory-management objectives and reduces unnecessary operational complexity.

Therefore, the final version should be understood as a complete, lightweight, and maintainable lab-management toolkit rather than a full enterprise infrastructure-management platform.

16. Conclusion

The Ansible-Based Linux/Ubuntu Lab Management Toolkit provides a practical, documented and professor-friendly approach to managing Ubuntu laboratory computers from a central control machine. It solves the main problem of repetitive manual administration while keeping the solution understandable and safe.

The final project includes a structured repository, eight Ansible playbooks, an interactive launcher, a logging mechanism, a central configuration file, controlled shared-material distribution, privacy-conscious Git practices and extensive documentation. It is suitable as an Erasmus internship deliverable and a foundation for future development by the department.

Final Conclusion

The toolkit is ready for real Ubuntu laboratory validation. After successful testing and completion of FINAL_TEST_REPORT.md, it can be handed over as a complete, practical and maintainable internship deliverable.

17. Skills Demonstrated and Professional Value

Beyond the immediate technical deliverable, the project demonstrates a set of practical skills that are directly relevant to software engineering, systems administration, DevOps and technical communication.

Skill Area	Evidence in the Project
Linux system administration	Management of Ubuntu PCs through SSH, package management, sudo checks, reboot checks and cleanup tasks.
Automation engineering	Reusable Ansible playbooks with a safe workflow and configurable behavior.
DevOps thinking	Separation of configuration, inventory, scripts, logs and documentation.
Security awareness	Private inventory handling, no committed secrets and clear warnings around reports/logs.
User-centered design	Professor-friendly menu, handover guides and beginner-readable documentation.
Testing mindset	One-PC-first validation, syntax checks and evidence collection in FINAL_TEST_REPORT.md.
Technical writing	Detailed guides for setup, customization, architecture, troubleshooting and handover.

Portfolio Value

This project can be presented as a practical DevOps and Linux automation portfolio project because it combines real infrastructure needs, automation code, safety design, documentation and validation methodology.

Appendix A. Command Reference

```
# Enter project folder
cd ~/ansible-ubuntu-lab-management

# Pull latest changes
git pull origin main

# Make scripts executable
chmod +x labmanage scripts/manage_lab.sh scripts/run_with_logging.sh

# Check Bash scripts
bash -n labmanage
bash -n scripts/manage_lab.sh
bash -n scripts/run_with_logging.sh

# Check Ansible syntax
ansible-playbook playbooks/00_preflight_check.yml --syntax-check
ansible-playbook playbooks/01_check_connection.yml --syntax-check
ansible-playbook playbooks/02_collect_lab_status.yml --syntax-check
ansible-playbook playbooks/03_update_system.yml --syntax-check
ansible-playbook playbooks/04_install_required_software.yml --syntax-check
ansible-playbook playbooks/05_copy_shared_materials.yml --syntax-check
ansible-playbook playbooks/06_clean_lab_computers.yml --syntax-check
ansible-playbook playbooks/07_reboot_if_required.yml --syntax-check

# Launch menu
./labmanage
```

Appendix B. Example Inventory

```
[students]
pc1 ansible_host=192.168.1.101 ansible_user=student
pc2 ansible_host=192.168.1.102 ansible_user=student
pc3 ansible_host=192.168.1.103 ansible_user=student

# Notes:
# - Replace IP addresses with real lab addresses.
# - Keep inventory.ini private.
# - Do not commit inventory.ini to GitHub.
```

Appendix C. Example Configuration

```
required_packages:
  - vim
  - git
  - curl
  - wget
  - python3
  - python3-pip
  - htop
  - tree

update_package_cache: true
upgrade_system_packages: true
autoremove_unused_packages: false

shared_materials_source: "shared_materials/"
shared_materials_destination: "/home/student/Lab_Materials"
shared_materials_owner: "student"
shared_materials_group: "student"

minimum_free_disk_gb: 2
minimum_memory_mb_warning: 2048

clean_package_cache: true
clean_apt_autoremove: true
clean_temp_files: false

reboot_timeout_seconds: 600
```

Appendix D. Final Acceptance Checklist

- ☐ GitHub repository updated
- ☐ MIT License added
- ☐ inventory.ini ignored and private
- ☐ config/lab_settings.yml present
- ☐ labmanage opens successfully
- ☐ Bash scripts pass syntax check
- ☐ All playbooks pass syntax check
- ☐ Preflight works on one PC
- ☐ Connection check works on one PC
- ☐ Status collection works on one PC
- ☐ Shared material copy verified remotely
- ☐ Install playbook verified on one PC
- ☐ Logging creates reports
- ☐ Safe checks run on all reachable PCs
- ☐ FINAL_TEST_REPORT.md completed